申

能申吗 • 政策资格匹配器

NENG SHEN MA · POLICY ELIGIBILITY MATCHER

贴一份政策原文，
30 秒知道自己够不够格。

把动辄上万字的资助政策原文，抽取成机器可逐条判定的条件树：
用户只回答一次「我是谁」，剩下交给确定性规则引擎 —— 每条结论都能点回原文那一句。

32	339	26	79
份国家级政策 原文逐字入库	条可判定条件 146 个条件组	项统一画像槽位 横向覆盖 32 份政策	项自动化测试 全部本地可复跑

参赛赛事 第九届「传智杯」全国大学生计算机大赛 · AI-WEB 赛道（团队赛 · B 组）
作品形态 Web 应用（Vue 3 + Express + DeepSeek），服务端部署与纯静态演示双模式
在线演示 https://someone227.github.io/shuanghuiwangzhongwang/
代码仓库 https://github.com/Someone227/shuanghuiwangzhongwang

文档版本 v1.0 · 编写日期 2026-10-27 · 对应代码提交 `6b13d6c`（main 分支）

本文档覆盖：系统架构、核心流程与时序、AI 调用链路、数据来源与风险控制、判定引擎设计、 质量保障，以及开发全过程的重大版本更新与缺陷修复记录。

目　录

01 项目概述：一个被上万字原文挡住的问题 …… 3

1.1–1.5 问题、定位、功能全景、使用场景、文档结构

02 系统架构：一次抽取，处处零 token 判定 …… 5

2.1–2.5 总体架构、技术选型、代码地图、双部署形态、数据流边界

03 核心流程与时序：从原文到判定的六步链路 …… 8

3.1–3.5 建库时序、判定时序、单条判定状态机、雷达与日历流程、失败与降级

04 AI 调用链路：把大模型关进「建库」这一步 …… 11

4.1–4.5 调用架构、提示词设计、结构化输出与校验、调用实测统计、留痕与人工复核

05 数据来源与风险控制：政策数据的可信边界 …… 14

5.1–5.5 来源清单、原文清洗、时效性处理、风险清单与对策、合规与隐私

06 判定引擎与横向能力：规则如何做到可解释 …… 16

6.1–6.5 条件模型、十种操作符、并列通道、四层结论、雷达与日历

07 页面体系与视觉设计：八个路由与两套主题 …… 18

7.1–7.4 路由与页面职责、交互一致性、双主题语义色板、响应式适配

08 质量保障：79 项测试与可复现的验证方法 …… 20

8.1–8.4 测试分层、数据契约测试、构建与部署流水线、可视化验收

09 重大版本更新记录 …… 22

9.1–9.7 七个里程碑：骨架与引擎 → 政策库扩容 → 账号体系 → 静态部署 → 横向能力 → 体验重构 → 决策回滚

10 缺陷修复记录 …… 25

10.1–10.4 判定正确性缺陷、体验与工程缺陷、修复方法论、回归保障

11 已知边界、风险与后续规划 …… 27

A 附录：接口清单、数据字典、提交统计 …… 29

阅读提示 **本文档的数据口径**

文中所有数字均由仓库代码与数据在 `2026-10-27` 实际统计得出，统计脚本随文档存放于 `docs/tech-report/`，可复跑复核。政策条款、金额、时间节点均以原文为准；本文档只描述系统如何判定， 不构成任何政策解释或申请承诺。

01 项目概述

OVERVIEW

1.1 问题：政策文本与「我能不能申」之间的缺口

学生资助、专项补贴、推免保研、基层就业代偿这类政策，几乎都有一个共同形态：**一份上万字的公文，加上若干个附件，真正的资格条件散落在条文、括号、脚注和「原则上」「一般应」这类弹性表述里。**本项目收录的 32 份国家级政策，原文合计 10.2 万字，被拆出 339 条可判定条件——平均每份政策 10.6 条，最多的《事实无人抚养儿童保障意见》有 23 条。

而用户的问题只有一个：**「我这种情况，到底能不能申？」**现行的解决方式都不理想：逐份读原文（读完仍不确定自己算不算「家庭经济困难」）、问辅导员和学长（口径因人而异）、直接去资助中心（跑了才知道缺材料）。信息不对称的成本，最终由最不熟悉流程的那部分学生承担。

痛点　政策文本不是给人「算」的

同一件事有 285 种字段写法：`education.level`、`student.educationLevel`、`education.stage`、`school.type`……跨政策复用极少，最高频的学历字段也只出现在 7 份政策里。逐份填一遍，等于把同一份简历抄 32 次。

机会　把「读」变成「算」

政策条文一旦被拆成「字段 + 操作符 + 阈值」的结构化条件，就可以像代码一样被逐条求值：给一份用户画像，引擎输出每条条件是符合、不符合还是待补充，并给出原文出处。判定这件事，本就不该交给概率模型。

1.2 产品定位：不是政策问答，是资格判定

市面上同类工具大多是「政策问答机器人」或「办事清单助手」：前者回答「政策说了什么」，后者列出「要准备什么」。它们都停在**信息整理**这一层，没有回答最关键的那一步——**以你个人的情况去逐条比对，你到底卡在哪一条。**本项目的差异点集中在三件事上：

维度	常见做法	能申吗的做法
输出结论	生成一段回答或一份清单，由用户自己对照	逐条给出「符合 / 不符合 / 待补充 + 建议」，并汇总为一句人话结论（可申 / 暂不符合 / 还不能下结论）
结论依据	模型生成，无法验证，可能编造	每条条件绑定 `docRef` 与原文 `quote`，页面可点回原文段落并高亮原句；测试断言引文 100% 可在原文中逐字定位
运行成本	每次提问调用大模型，结果不稳定、有 token 成本	大模型只在开发期「建库」用一次；用户每次判定都是本地确定性引擎，零 token、断网可跑、结果可复现

1.3 功能全景

产品把「贴政策 → 判自己」拆成六个可验证的步骤。前四步发生在开发期与入库期，后两步是用户每次使用都会走的链路：

#	步骤	做什么	谁执行	对应页面 / 产物
01	政策入库	从教育部、全国学生资助管理中心等官网抓取原文，逐字存档不加工	人工 + 脚本	`data/policies/raw/*.txt`
02	AI 抽条件	DeepSeek 把原文抽成「条件组 / 条件 / 字段 / 材料 / 时间节点」的 JSON 条件树	DeepSeek `deepseek-chat`	`data/policies/*.json` + `docs/ai-log/`
03	人工复核	逐条核对条款、引用、判定口径，修正模型输出的分类与逻辑错误，过时条款写进 notes	团队（57 条复核记录）	`review.changes[]`
04	填自己	表单字段由政策动态生成，只问这份政策真正用得上的信息；留空不算不符合	用户	`/match/:id`
05	本地判定	规则引擎在服务端（或浏览器内）逐条求值，输出四类状态与汇总结论	判定引擎	`matcher.ts` / `evaluate()`
06	回溯原文	每条结论可点回原文段落，命中的句子高亮；附材料清单与时间节点	用户	判定报告 + 政策原文标签页

表 1-1　六步链路：前四步保证「条件是对的」，后两步保证「结论是可查的」。

围绕这条主链路，系统还长出了四个横向能力，解决主链路之外的三个问题：**不知道该看哪份**（政策雷达：26 项画像横向判定 32 份）、**不知道什么时候申**（年度申报日历：60 个时间节点按月摊开）、**想把条件看明白再填**（政策详情页：条件树 / 材料 /

时间节点 / 原文一页读完）、 **想换设备接着填**（账号体系与个人中心）。

1.4 一个真实的使用场景

场景 **大二学生小李，想知道自己能申哪些钱**

她打开政策库，32 份政策一眼望不到底。于是点开**政策雷达**，只填了 19 项共性信息（学历层次、年级、入学年份、学校类型、 成绩排名、家庭是否困难……）。页面当场给出分层结论：**4 份初筛适合、11 份差几项、2 份信息不够、15 份不符。**

她点进「国家奖学金」，判定页已经从画像带入了 8 项信息（可改），剩下的几项按提示补上。点「开始判定」， 13 条条件逐条列出：11 条符合、2 条待补充，并明确写出「成绩排名前 10% 属于常规通道；若排名在前 10%–30% 之间， 需要同时提供其他方面表现特别突出的证明材料」——这句话直接对应原文第七条（二）。

她顺手点开**申报日历**：10 月有 4 个节点，其中「学生向学校提出申请」标着**「个人要办」**。整个流程没有一个字是模型现编的。

1.5 文档结构

本文档按「问题 → 架构 → 流程 → AI → 数据 → 引擎 → 界面 → 质量 → 演进 → 缺陷 → 边界」组织。 其中**第九章（重大版本更新）与第十章（缺陷修复记录）**是本文档的重点：它们记录了项目从空仓库到当前版本的全部关键决策， 包括三次判断失误与一次整体回滚。**系统如何工作**是前八章的内容，**它为什么会变成现在这样**是后三章的内容——后者同样重要， 因为一个 32 份政策、339 条条件的判定系统，真正的风险从来不在代码写不出来，而在「判定口径悄悄错了却没人发现」。

02 系统架构

ARCHITECTURE

一句话概括架构：**大模型只在开发期出现一次，把「自然语言政策」编译成「结构化条件树」； 运行期是一套纯函数规则引擎，同一份代码既跑在 Node 服务端，也直接打包进浏览器。** 这个选择决定了整个系统的成本结构、可测试性与可解释性。

2.1 总体架构

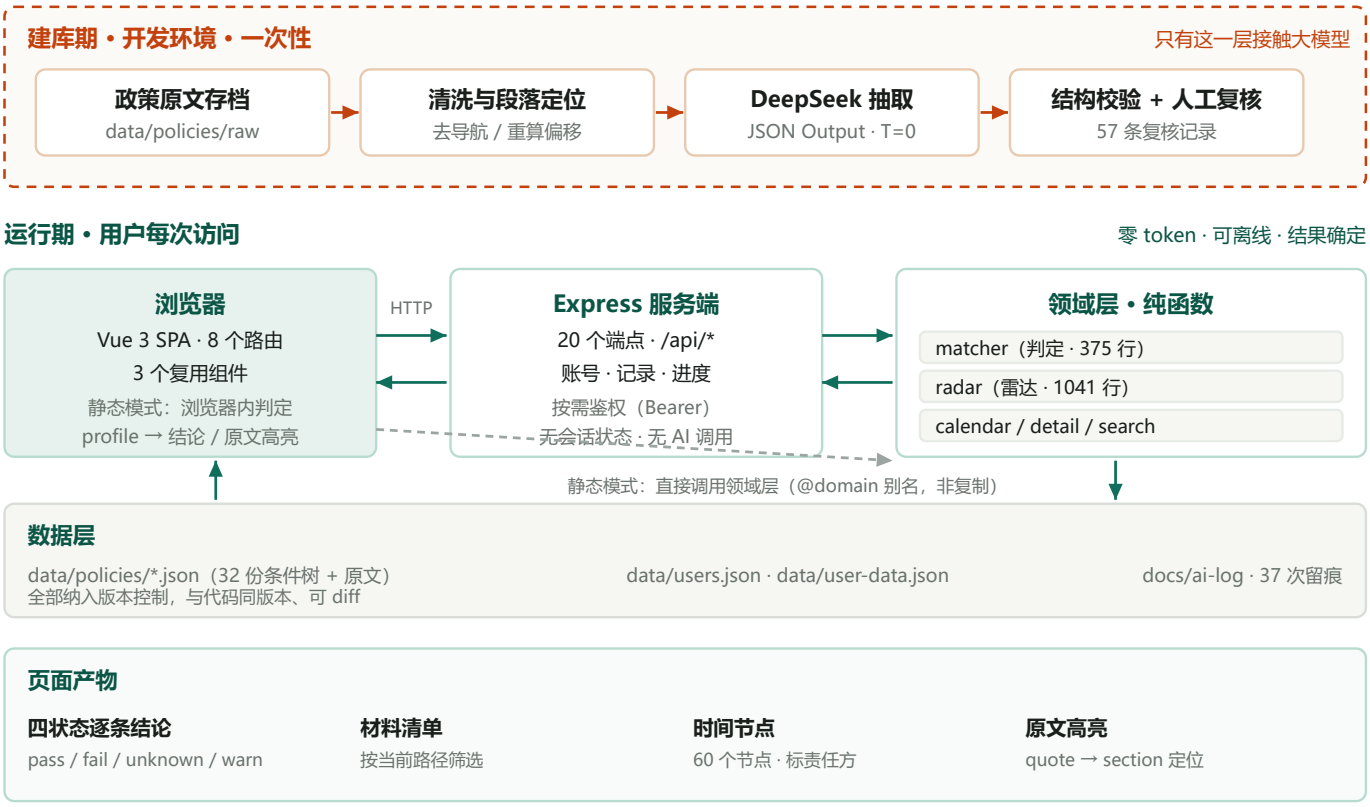


图 2-1 总体架构：建库期（有 AI）与运行期（零 AI）之间只有构建产物一条通路。

架构可以分成三条独立的链路，它们之间只通过文件与 HTTP JSON 耦合：

① 建库链路（开发期）
抓取原文 → 清洗噪声 → DeepSeek 抽取 → 结构校验/重试 → 人工复核 → 提交入库。 产物是 `data/policies/*.json`，随仓库走版本控制。

② 判定链路（运行期）
页面拉取政策 JSON，动态生成表单；用户填表后调用 `evaluate()`， 纯函数返回逐条结论。全程无网络、无模型、无密钥。

③ 账号链路（可选）
只用于跨设备保存「判定记录」与「填到第几项」，不保存填报内容本身。 静态部署模式下整条链路不启用。

三条链路里，只有 ① 需要 `DEEPSEEK_API_KEY`，而它**永远不在运行期被调用。** 这一点在代码里有硬约束：前端不携带、也不请求任何模型密钥；服务端除抽取脚本外没有任何 AI SDK 依赖。

2.2 技术选型与理由

层	选型	版本 / 位置	为什么是它
前端	Vue 3 + Vite + TypeScript + Tailwind CSS v4	`web/`	SPA 路由与组合式 API 适合「多页面共享同一画像」；Tailwind v4 的 `@theme` 让双主题只靠 CSS 变量切换
后端	Node.js + Express 4 + TypeScript（tsx 直跑）	`server/src/`	与前端共用 TypeScript 类型；无需构建步骤即可本地起服务，评委拿到仓库 `pnpm dev` 就能跑

层	选型	版本 / 位置	为什么是它
数据	JSON 条件树（随仓库提交），无数据库	`data/policies/*.json`	32 份政策是只读数据集，JSON 可直接 diff、直接 review；静态部署时原样导出成静态文件
判定引擎	手写纯函数（无依赖）	`server/src/domain/matcher.ts`（375 行）	纯函数才能同时跑在 Node 与浏览器，也才能被 79 项测试逐条钉住行为
大模型	DeepSeek `deepseek-chat`（JSON Output，temperature 0）	`server/scripts/`	只在建库期调用；JSON Output + 严格校验，把「不可控的生成」压成「可校验的结构」
账号	scrypt 加盐口令 + 无状态 HMAC-SHA256 令牌	`server/src/auth/`	不引入数据库与会话表，单机 JSON 足够；令牌无状态，重启不掉线
测试	`tsx --test`（Node 内置 test runner）	`**/*.test.ts` 共 9 个文件	零额外依赖，`pnpm test` 一条命令复跑；测试即判定口径的书面规范
部署	Express 服务端模式 + GitHub Pages 纯静态模式	`tools/deploy-pages.mjs`	机房电脑装不了环境时，评审点开链接就能用；静态模式与后端模式共用同一套判定引擎

表 2-1 技术选型。每一项都服务于同一个约束：**评委不装环境、不配 Key 也能完整验收**。

2.3 代码地图

目录	文件数	行数	职责
`server/src`	28	4958	领域层（判定引擎、雷达、日历、检索）、存储层、鉴权、路由
`web/src`	22	4846	8 个页面视图、3 个通用组件、API 封装、登录态、主题
`server/scripts`	6	868	抽取、清洗、偏移修复、复核落地、静态数据导出
`tools`	2	389	GitHub 同步工具、gh-pages 发布脚本
`data/policies`	64	—	32 份条件树 + 32 份原文（`raw/`），共 10.2 万字原文
`docs/ai-log/policies`	32	—	每份政策一次抽取的完整请求 / 响应 / token 用量

表 2-2 代码规模（`2026-10-27` 统计）。核心判定逻辑只有 375 行，但被 12 个测试用例单独覆盖。

2.4 两种部署形态：同一套引擎，两个宿主

纯函数引擎带来一个直接好处：**它不需要服务端也能跑。**前端在 `vite.config.ts` 里把 `@domain` 别名指向 `server/src/domain`，静态构建时这套领域代码被打进浏览器 bundle；服务端模式则通过 HTTP 调用。 两条路径的判定结果**逐字段一致**——因为执行的是同一份源码。

能力	服务端模式（`pnpm dev` / 自建服务器）	静态模式（GitHub Pages）
政策数据	Express 读 `data/policies/*.json`，`GET /api/policies`	构建期导出成静态 JSON，浏览器 `fetch`
判定执行	服务端 `POST /api/policies/:id/evaluate`	浏览器内直接调用 `evaluate()`
政策雷达 / 日历	`GET /api/radar`、`GET /api/calendar`（服务端现算）	预先导出的 `radar.json`、`calendar.json`
账号与判定记录	✅ 注册登录、跨设备记录、填报进度	❌ 无账号体系（其余功能完全一致）
密钥	仅建库脚本需要	完全不需要

设计取舍 **为什么用「导出静态数据」而不是在浏览器里重新实现一遍逻辑**

重写一遍意味着两份实现、两套 bug。这里的做法是把**同一份领域代码**搬进浏览器：`matcher.ts`、`radar.ts`、`calendar.ts` 都是无 Node 依赖的纯 TS，`@domain` 别名一行配置即可复用。 代价是静态构建要显式导出数据（`export-static-data.mjs`），换来的是「判定行为只有一份定义」。

2.5 数据边界：什么数据存在哪里

数据	存放位置	是否入库	说明
政策原文 / 条件树	`data/policies/`	✅	只读数据集，可 diff、可 review、可回滚

数据	存放位置	是否入库	说明
AI 调用日志	`docs/ai-log/policies/`	✅	含 messages / response / usage，**不含请求头与 Key**
账号与口令哈希	`data/users.json`	❌（gitignore）	scrypt 加盐，首次启动幂等创建演示账号
判定记录 / 填报进度	`data/user-data.json`	❌（gitignore）	只记「判定过哪份、填了几项」，**不记填了什么**
表单填报内容	页面内存	❌	离开或刷新即清空，浏览器与服务端都不落盘
画像草稿	浏览器 localStorage	❌	只存在用户本机，不上传
模型密钥	`.env`	❌（gitignore）	仅建库脚本读取，运行期代码不引用

表 2-3 数据边界。判定涉及成绩排名、家庭经济状况等敏感信息，因此「填报内容不落盘」是硬约束， 由提交 `11f3a55` 显式实现并在 README 中向用户明示。

03 核心流程与时序

FLOW & SEQUENCE

这一章回答三个问题：**一份政策是怎么变成可判定的 JSON 的**（建库时序）、**一个用户点「开始判定」之后发生了什么**（判定时序）、**单条条件和一个条件组是怎么得出结论的**（状态机）。

3.1 建库链路：一份政策的完整生命周期



一次建库的实测指标

32 份	37 次	219,952	12.8 s	< 1 元
政策入库	调用（5 份重试）	总 token	平均单份耗时	一次建库成本

图 3-1 建库链路五步：原文 → 清洗 → 抽取 → 校验 → 人审，每一环都有可核查的产物。

建库链路由 6 个脚本串成，每一步都留下可复核的产物：

脚本	行数	职责与关键约束
`clean-policies.mjs`	162	截掉网页导航与页脚噪声，按固定正文起点重算段落偏移；正文条款一字不改
`extract-policies.mjs`	314	调用 DeepSeek 抽取条件树；**结构校验不通过则带错误重试一次**；请求/响应写入 ai-log
`repair-sections.mjs`	92	修复段落切分与偏移（模型摘出的引文必须能在原文中定位）
`fix-offsets.mjs`	57	归一化定位：把 `start/end` 偏移重算成可稳定复现的位置
`apply-review.mjs`	149	把人工复核结论（改分类、调硬性、补 notes、删过时条款）批量落到 JSON，并写回 `review.changes`
`export-static-data.mjs`	94	导出 `index.json` / `radar.json` / `calendar.json` 供静态部署使用

表 3-1 建库脚本。6 个脚本共 868 行，全部是一次性工具，不进入运行期依赖。

3.2 判定时序：从点下按钮到看见结论

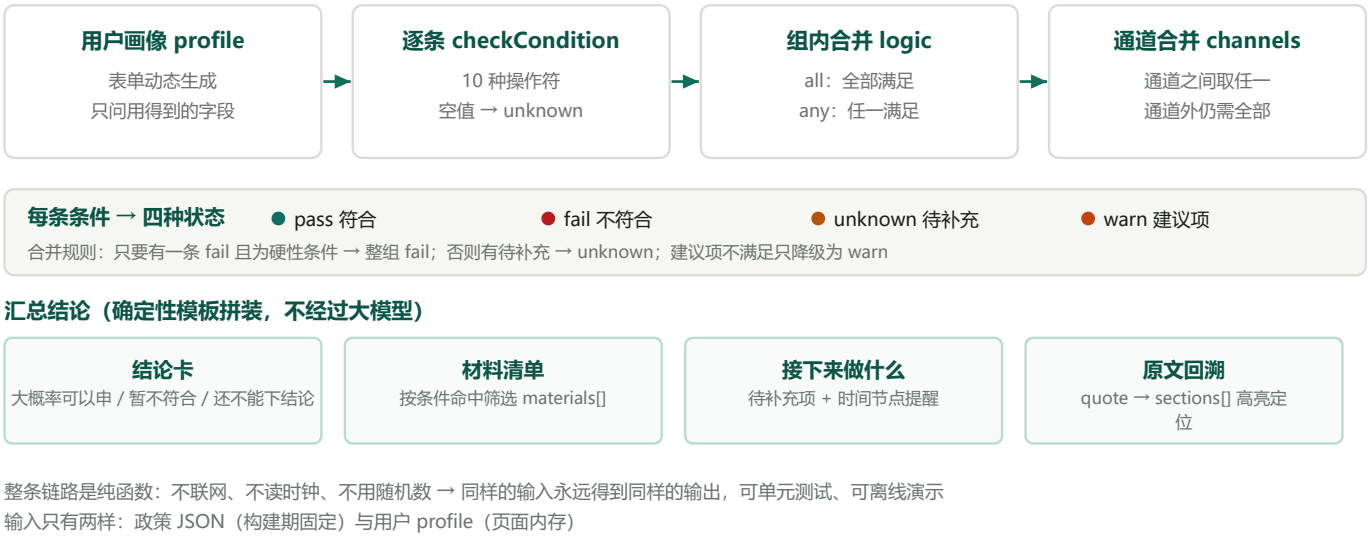


图 3-2 判定数据流：profile 经逐条求值、组内合并、通道合并，输出结论与证据。

一次判定在服务端模式下涉及 4 个参与者、6 次交互。注意两点：**判定请求体只带画像、不带政策**（政策由服务端从已入库 JSON 读取，避免前端篡改数据）；**返回值包含全部证据链**（每条条件的 `quote`、`docRef`、`gap`、`hint`），前端只负责渲染，不做任何判定逻辑。

步	交互	数据形态
1	进入 `/match/:id`	路由参数 `id`；页面并发拉取政策全量 JSON 与当前画像草稿
2	动态渲染表单	按 `policy.fields[]` 生成字段，按 `lib/field-groups.ts` 的语义分组折叠
3	用户填表	画像为扁平的 `Record<string, unknown>`，键是 `education.grade` 这类点路径
4	点击「开始判定」	`POST /api/policies/:id/evaluate`，body = `{ profile }`
5	引擎求值	`evaluate(policy, profile)`：逐组求值 → 通道裁决 → 汇总 → 生成 headline / 材料清单 / 下一步
6	渲染报告	状态色块 + 逐条卡片 + 原文标签页；每条卡片可跳原文并高亮 `quote`

3.3 单条条件与条件组：四种状态与两种合并规则

判定引擎只产出四种状态，含义被严格定义——**用户最容易被误伤的，就是「没填」被当成「不符合」**：

状态	标记	含义	参与汇总结论
pass	符合	用户填的值满足该条件	✅
fail	不符合	用户填的值明确不满足（且该条件是硬性）	✅
unknown	待补充	字段没填 / 值无法与政策选项对齐 —— **不知道，不等于不符合**	✅（作为「还不能下结论」的依据）
warn	建议项	非硬性条件不满足，只提示不影响结论	❌

合并规则按条件组的 `logic` 分两种，另有两条兜底：

规则 **合并逻辑**

all（全部满足）：出现 fail → 组 fail；否则出现 unknown → 组 unknown；否则 pass。
any（满足任一）：出现 pass → 组 pass；否则全为 fail → 组 fail；否则 unknown。
兜底 1：any 组内条件全是建议项（`required:false`）时，不把整组判为 fail —— 否则「择优参考」会把用户误判成不符合（提交 `5c4a0a6`）。
兜底 2：配置了并列通道的政策，先按通道分组裁决，**任一通道 pass 即整体 pass**；通道之外的条件组仍需全部满足。

最终 `headline` 由 `buildHeadline()` 生成，只有三种口吻，且都带前提说明：**「大概率可以申」**（硬性条件全 pass）、**「暂不符合」**（存在硬性条件 fail）、**「还不能下结论」**（存在待补充）。判定报告页尾部固定一行提示：结论只反映你填写的信息，最终是否通过仍以学校、主管部门审核为准。

3.4 横向能力：政策雷达与申报日历

政策雷达：一次画像，32 份分层

32 份政策共 333 个表单字段、285 种写法，逐份填不现实。雷达只问 26 项**跨政策复用**的信息， 用规则表把它们映射到每份政策自己的字段 key 上，再逐份调用同一个 `evaluate()`：

1. **槽位定义**：26 项 `RADAR_SLOTS`，带 `aliases`（同一件事的不同叫法）
2. **字段映射**：`FIELD_RULES` 把槽位值翻译成政策字段 key；映射不上**宁可不写**
3. **逐份判定**：调用 `matcher.evaluate`，与单政策页完全同源
4. **四层分层**：初筛适合 / 差几项 / 信息不够 / 不符

申报日历：60 个节点按月份摊开

原文里的时间表述是自然语言（「每年 10 月 31 日前」「6 月底」「入学后两周内」）， `parseDeadlineText()` 把它们解析成可排序的结构：

1. **解析**：年度周期 / 具体日期 / 学期节点 / 办理时限 / 政策沿革 / 其他
2. **归月**：能落月的进 12 个月份格，落不进的单独归类
3. **责任方**：按主语 + 动作判断「个人要办 / 学校院系 / 主管部门」
4. **时效**：往年一次性节点不混进今年待办，单独折叠为「政策沿革」

3.5 失败路径与降级

失败场景	系统行为	设计理由
政策数据加载失败	页面显示可读错误，不阻塞其他路由	静态部署下单个 JSON 404 不应让整站白屏
画像字段未填	判为 `unknown`（待补充），不算不符合	漏填被当成不符合，会让用户放弃本来能申的项目
画像值与政策选项对不上	字段标「请选择」并为空 → `unknown`；雷达映射不上则跳过该项	写错比留空伤害更大
模型输出结构不合法	建库期校验失败 → 带错误重试一次；仍失败则该政策不入库	宁缺毋滥：坏数据入库比少一份政策更糟
服务端不可用	账号相关功能降级；判定可切静态模式继续用	答辩/评审现场网络不可控
深链直接访问	静态部署用 `404.html` 回退 + `BASE_URL` 子路径	解决历史模式路由在 GitHub Pages 刷新 404（提交 `5299987`）

表 3-2 失败路径。核心原则：**宁可说「不知道」，也不要把不确定说成「不符合」。**

04 AI 调用链路

AI PIPELINE

这个项目对「AI 用在哪」有一条硬边界：**大模型只在开发期把政策原文编译成结构化条件树，运行期一次也不调用。** 好处不是省 token，而是把「不确定的生成」隔离在可人工复核的建库阶段，让用户每次看到的结论都是确定、可复现、可解释的。

4.1 调用边界：一道运行期防火墙

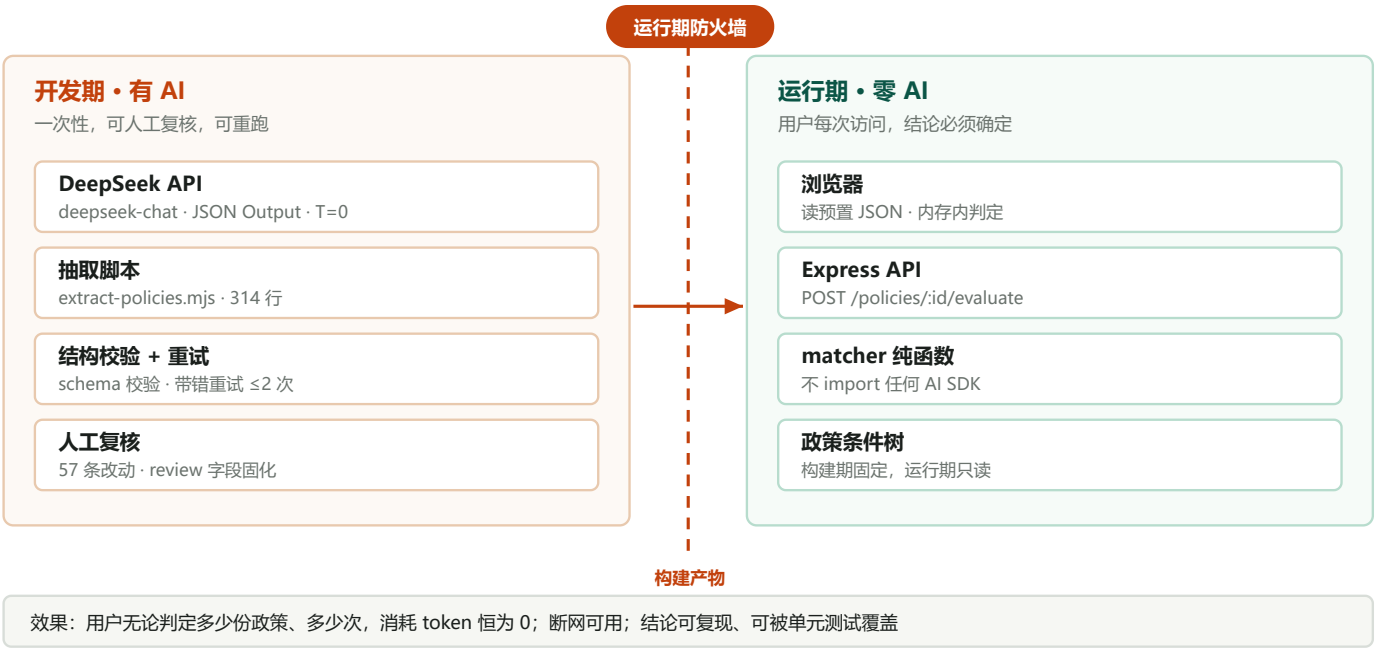


图 4-1 AI 调用边界：大模型只出现在左侧开发期，右侧运行期不 import 任何 AI 依赖。

完整链路只有 4 个节点、7 类产物，全部落在开发期：

#	节点	输入	输出 / 去向
1	原文抓取与存档	官网页面	`data/policies/raw/*.txt`（逐字存档，不加工）
2	噪声清洗	原始网页文本	去除导航/页脚，重算段落偏移；`cleaning` 字段记录移除量
3	DeepSeek 抽取	system prompt + 政策原文	条件树 JSON（结构校验通过才落盘）
4	人工复核	条件树 + 原文	`review.changes[]`（57 条复核记录）+ 修正后的 JSON

表 4-1 建库期调用链。所有节点都是一次性脚本，运行期代码不 import 任何 AI SDK。

4.2 提示词设计：把「解释政策」改成「编译政策」

系统提示词只有 9 条要求，但每一条都对应一个具体的失败模式。核心思路是**先约束输出形态，再约束内容质量**：

#	提示词要求（原文摘要）	它防的是什么
1	只输出 JSON，不要 markdown 代码块、不要解释文字	模型「先解释再输出」，导致解析失败
2	每条条件必须回链原文：`docRef` 指向 section.id，`quote` 必须是原文句子（可截取不可改写）	幻觉条款：凭空编一条原文里没有的条件
3	条件必须可判定：`field` 点路径、`operator` 枚举、`value` 阈值/枚举	输出「视情况而定」这类无法执行的软结论
4	主观条件也抽成可判定条件，由用户自填是/否，**不设 autoUncertain、不编造规则**	模型自作主张替用户判断「家庭经济困难」
5	非硬性条件设 `required:false`，硬性门槛设 `required:true`	把「择优参考」当成硬门槛，误伤用户
6	组内 `logic` 取 all/any，多组默认顺序全部满足	把「任一满足」写成「同时满足」，导致全员不符合
7	忽略网页导航、页脚、联系方式	把「扫一扫分享本页」抽成政策条款
8	字段值用原文中文表述（全日制/本科/硕士），不擅自翻译成英文或数字	枚举值与表单选项对不上，判定永远 unknown

#	提示词要求（原文摘要）	它防的是什么
9	`hints` 面向用户：不满足时给一条可操作建议	给出结论但不告诉用户下一步做什么

表 4-2 九条提示词要求与对应风险。提示词全文见 `server/scripts/extract-policies.mjs` 的 `SYSTEM_PROMPT`。

一次请求的完整形态

```
// server/scripts/extract-policies.mjs  callDeepseek()
POST {BASE_URL}/chat/completions
{
  model: "deepseek-chat",          // 可用 DEEPSEEK_MODEL_CHAT 覆盖
  messages: [system, user],        // system = 九条要求；user = 元信息 + 原文
  temperature: 0,                  // 抽取任务不需要创造性
  max_tokens: 8000,
  response_format: { type: "json_object" }   // 强制 JSON Output
}
// 超时 240s（AbortController）；请求体与响应体原文写入 docs/ai-log/policies/<id>.json
```

4.3 结构化输出的三道闸门

模型输出不可能「一次就对」，所以建库脚本设计了三层防御，任何一层不通过都不会污染数据集：

① 协议层

`response_format: json_object` 强制 JSON；解析器额外兼容 ```` ```json ```` 围栏，避免模型习惯性包裹代码块。

② 结构层

`validatePolicy()` 校验 9 类必填结构：id 一致、sections/conditions 非空、field.type 与 operator 在枚举内、**docRef 必须真实存在于 sections**、logic 只能是 all/any。

③ 修正层

校验失败时，把「上一次模型输出」作为 assistant 消息、把错误清单作为下一条 user 消息，要求模型**修正后重新输出**；最多 2 次尝试，仍失败则该政策不入库。

三层闸门的效果可以从日志里直接验证：**37 次调用中，5 份政策在第 1 次未通过校验，带错误修正后全部通过，没有一份被丢弃。** 换句话说，最终入库的 32 份政策，每一份都通过了完整的结构校验。

4.4 实测统计：一次建库的完整成本

37 次
DeepSeek 调用
32 份政策 / 5 份重试

219,952
总 token
prompt 113,422 + completion 106,530

12.8 秒
平均单份耗时
首请求 → 产物落盘

0 次
运行期调用
用户判定不消耗 token

指标	数值	说明
政策份数 / 调用次数	32 / 37	5 份政策首次输出未通过结构校验，追加一轮修正
prompt / completion token	113,422 / 106,530	prompt 主要为政策原文（单份 2,000–6,000 字）
请求 / 响应体字符数	282,035 / 399,491	全部存档，可逐字回放
平均单份耗时	约 12.8 s	含网络往返与 JSON 生成；最长的是多轮修正的政策
缓存命中 token	0	每次请求都带完整原文，无缓存复用
运行期 AI 成本	0	用户无论判定多少次，都不产生模型调用

表 4-3 建库成本（`docs/ai-log/policies/*.json` 全量统计）。按 DeepSeek 官方定价，一次完整建库的成本在**人民币 1 元以内**——这意味着新增一份政策、重跑一次抽取，几乎没有成本顾虑。

4.5 留痕与人工复核：AI 只是「初稿」

每份政策的 AI 调用都完整留痕，字段结构固定：

```
// docs/ai-log/policies/<id>.json
{
  "id": "national-scholarship",
  "title": "本专科生国家奖学金评审办法（教财函〔2019〕105号）",
  "source": "http://www.moe.gov.cn/...",
  "generatedAt": "2026-10-25T09:33:18.613Z",
  "logs": [{
    "attempt": 1,
    "request": [{role:"system",...},{role:"user",...}],   // 完整 messages
    "responseRaw": "{...}",                                // 原始响应，含 usage
    "usage": {prompt_tokens:2476, completion_tokens:3506, total_tokens:5982},
    "model": "deepseek-flash"                              // 服务端回显的模型快照名
  }]
}
```

抽取只是初稿。32 份政策共产出 57 条人工复核记录，全部写进 `review.changes[]`，按性质大致分为：

复核类型	约数	典型记录（原文引用）
新增收录逐条核对	20	「逐条核对原文条款、引用与字段，确认条件可判定、引文可回溯」
判定口径调整	约 14	「退役后复学」与「在校入伍补偿」是并列情形，退役复学一项改为建议项，避免在校生被误判
时效性处理	约 9	notes 追加【现行标准】：2024 年起本专科国奖由 8000 元提高到 10000 元（财教〔2024〕181号）
判定逻辑修正	3	「年龄条件」组同时写了硕士（≤40）与博士（≤45），all 逻辑下要求学历同时为硕士和博士 → 拆成两条并列通道
分类与引文修正	约 7	category 由模型输出的「其他」改为「困难认定」；c4 的原文引用补全括号说明，确保整句可逐字回溯
逐条核对条件与引用	4	「身份（全日制硕士/博士）、政治品行、学业科研三项为硬性条件」

表 4-4　57 条人工复核记录的分类。它们既修正了模型错误，也把「判定口径」写成了可追溯的书面依据。

关键教训 **模型最危险的不是编造条款，而是「看起来对」的逻辑组合**

三条「判定逻辑修正」有一个共同点：**每一条单独看都忠实于原文，但组合起来就错了。**例如少数民族骨干计划原先把「报考硕士（年龄 ≤ 40）」与「报考博士（年龄 ≤ 45）」写进同一个 all 组——对任何一个人，都不可能同时是硕士和博士，于是这份政策对所有人判为不符合。 这类错误不会被 JSON 结构校验发现，只能靠人读原文、对照判定结果才会暴露。这也是为什么项目坚持「AI 抽完必须人工复核」的原因。

05 数据来源与风险控制

DATA & RISK

> 政策判定系统的风险，八成不来自代码，而来自**数据是否真实、是否时效、是否被误读。** 这一章把「数据从哪来、怎么处理、可能错在哪、错了怎么办」全部摊开。

5.1 来源：只取官方原文

32 份政策全部来自官方渠道，每份 JSON 都保留 `sourceUrl`，页面详情页可一键跳转原文：

来源机构	政策数	覆盖范围（部分）
教育部（moe.gov.cn）	约 20	国家奖学金、励志奖学金、助学金、推免、公费师范、免费学前、义务教育补助、营养改善、中职资助等
全国学生资助管理中心（xszz.edu.cn）	约 8	国家助学贷款、勤工助学、基层就业代偿、入伍资助、困难认定等
财政部 / 民政部 / 人社部 / 国务院	约 4	低保、特困人员供养、残疾人两项补贴、临时救助、就业见习岗位募集

表 5-1 来源分布。所有原文逐字存档于 `data/policies/raw/`，与条件树同仓库、同版本、可 diff。

强调一点：**不抓第三方转载、不抓公众号、不用模型「凭记忆」补全条款。**原文是唯一事实来源， 条件树里每一条 `quote` 都能在 `raw/` 的对应文本中逐字定位——这一条由测试断言（见 8.2）。

5.2 清洗：只删网页噪声，不动条款

直接抓下来的网页文本会带上一大堆与政策无关的内容。以国家奖学金通知为例，抓取文本的开头是「登录 / Languages / English / 日本語 / 无障碍浏览」， 结尾是「扫一扫分享本页 / 责任编辑 / 网站声明 / 京ICP备…」。这些内容如果混进原文，会被模型误抽成条款。

指标	数值	说明
有清洗记录的政策	7 份	其余 25 份原文本身干净，无需截断
累计移除字符	2,623 字	全部为导航、页脚、分享按钮、版权信息
单份最大移除	507 字	推免办法（网页结构最复杂）
正文改动	0	清洗只截头尾，条款一字未改；段落偏移已重算

表 5-2 原文清洗统计。每份政策都记录 `cleaning.removedChars` 与 `removedFrom`，可复核。

5.3 时效性：过时条款不参与判定

政策库里有相当一部分文件是「老办法」——奖励金额、贷款额度、试点范围早已被新文件替代。直接按原文判定会给出错误建议， 因此系统采用统一策略：**过时的数值/范围不进条件树，改写成 `notes` 里的【现行标准】提示，并在判定页显式展示。**

条款	原文（过时）	现行标准（notes 提示）
国家奖学金金额	每生每年 8000 元、名额 6 万名	2024 年起 **10000 元/年**、名额 12 万名（财教〔2024〕181号）
国家励志奖学金	每生每年 5000 元	2024 年起 **6000 元/年**（财教〔2024〕181号）
国家助学金	平均 3300 元/年	2024 年秋季起平均 **3700 元/年**，区间 2500–5000 元（财教〔2024〕181号）
助学贷款额度	每生每年 6000 元	现行本专科 **20000 元**、研究生 **25000 元**
基层就业代偿上限	每生每年 6000 元	2009 年标准，现行额度以当年通知为准
研究生国奖金额	硕士 2 万元 / 博士 3 万元（2012 年办法）	以当年通知为准
贷款试点省份	2008 年试点省份名单	已全国推开，试点条款不再作为条件

表 5-3 过时条款处理。原则：**不当有效条件用，但要告诉用户「为什么你看到的原文金额和现在不一样」。**

5.4 风险清单与对策

风险	等级	表现形式	已有对策
政策更新导致结论过期	高	库里的政策被新文件替代，页面仍按旧条件判定	每份带 `year` 与 `sourceUrl`；notes 标【现行标准】；判定页提示「以学校/主管部门当年通知为准」

风险	等级	表现形式	已有对策
模型幻觉条款	高	抽出原文没有的条件	`docRef + quote` 强制回链；结构校验；测试断言 100% 引文可定位；人工复核 57 条
逻辑组合错误	高	单条正确、组合后把人判错（见 4.5）	并列通道模型；多轮人工核对；针对这类错误补了专门测试
字段语义漂移	中	同一字段在不同政策里含义不同（入学年份 vs 年级）	字段 key 用点路径语义化；触发过真实 bug（见 10.1）并加了回归测试
覆盖不全 / 非全国	中	只有 32 份国家级政策，地方细则差异大	页面与文档明示「国家级政策库」；地方差异写入 notes
隐私泄露	高	填报内容含成绩排名、家庭经济状况	填报内容不落盘（仅页面内存）；账号只存判定记录；画像草稿只在本机 localStorage
静态版数据陈旧	中	gh-pages 是构建期快照	发布脚本每次重新导出数据；页面标注构建信息
用户误读为官方结论	中	把「大概率可以申」当审核通过	结论措辞克制（「初筛」「大概率」）；报告页固定免责声明；hints 写明线下核验环节

表 5-4 风险清单。高等级风险全部有对应工程手段，而不是只写在免责声明里。

5.5 合规、隐私与免责

- **个人信息最小化**：表单只问这份政策真正用得到的字段；不收集手机号、身份证号等无关信息。
- **填报内容不保存**：判定页的表单只存在页面内存里，离开或刷新即清空；服务端只记录「判定过哪份政策、填了几项」。
- **密钥不入库**：`DEEPSEEK_API_KEY` 只放 `.env`（已 gitignore），运行期代码不引用；AI 日志不含请求头与 Key。
- **不构成承诺**：判定结果只是基于用户自填信息与政策原文的初筛，最终是否通过以学校与主管部门审核为准；页面与文档均显式声明。
- **来源可溯**：每份政策保留发布单位、发文字号、原文链接、年份与原文存档，用户可自行核对。

06 判定引擎与横向能力

RULE ENGINE

判定引擎是这套系统真正「不可替代」的部分：375 行纯函数，10 种操作符，4 种状态，2 种合并规则， 支撑起 32 份政策 339 条条件的逐条求值。它不联网、不读时钟、不依赖随机数——同样的输入永远得到同样的输出。

6.1 条件数据模型

字段	类型	作用
`id`	string	条件组 / 条件唯一标识，用于通道引用与结果回传
`name`	string	组名（如「基本条件」「成绩要求」），页面按组折叠展示
`logic`	'all' | 'any'	组内合并规则：全部满足 / 满足任一
`field`	string（点路径）	用户画像的取值路径，如 `education.grade`
`operator`	枚举（10 种）	比较方式，见 6.2
`value`	string | number | boolean | string[]	阈值或枚举值；支持 `{{field}}` 模板引用其他字段
`required`	boolean	硬性门槛 / 建议项。建议项不满足只提示，不影响结论
`docRef` + `quote`	string	原文出处与原文句子，构成证据链
`hint`	string?	不满足时的可操作建议

表 6-1 条件结构。所有字段都由 AI 初稿 + 人工复核产出，不允许运行期修改。

6.2 十种操作符

操作符	语义	典型用法（来自真实政策）
`eq` / `ne`	等于 / 不等于（字符串归一化后比较，数字可跨类型比较）	国籍 = 「中华人民共和国」；是否在职 ≠ 「是」
`in` / `notIn`	属于 / 不属于某个集合	学历层次 in [「本科」「专科」]
`gte` / `lte` / `gt` / `lt`	数值比较（含边界）	年级 ≥ 2；成绩排名 ≤ 10（%）
`contains`	文本包含 / 多选包含	特殊群体包含「最低生活保障家庭学生」
`exists`	是否提供了该项信息	是否持有有效《残疾人证》

表 6-2 操作符一律是白名单枚举，模型输出不在枚举内会被结构校验打回。

6.3 三个容易被忽略的实现细节

① 模板比较

条件值可以写成 `{{education.grade}}`，求值时替换为另一个字段的值 —— 用于「毕业年份不早于入学年份 + 学制」这类字段间约束。 模板引用的字段没填时，该条件返回 `unknown` 而不是 fail。

② 空值语义

`isEmpty()` 把空串、空数组、null、undefined 统一视为「没填」； 未填条件一律 `unknown`。这条规则是整个产品体验的底线：**留空不会被算成不符合。**

③ 归一化比较

`normalize()` 去空格、统一大小写、全半角，`toNumber()` 兼容「10%」「10 名」这类带单位的写法 —— 避免「本科」与「本科 」被当成两个值。

6.4 并列通道：一份政策里的「或者」

真实政策里大量存在「满足 A 或 B 即可」的结构。如果全部压成 all，就会把符合条件的人判成不符合。 系统用 `channels` 显式表达这种并列关系：**通道内条件仍需满足，通道之间满足任一即可，通道之外的条件组照常全部满足。**

政策	通道	来源条款
本专科生国家奖学金	常规成绩通道 / 突出表现通道	第七条（二）：排名前 10% 直接申请；前 30% 且其他方面特别突出亦可
中职国家奖学金	前 5% 直接认定 / 前 30% + 突出表现	评审办法中的两档成绩要求

政策	通道	来源条款
少数民族骨干计划	报考硕士研究生 / 报考博士研究生	年龄限制按报考层次分别为 ≤40 与 ≤45

表 6-3 3 份政策、6 条并列通道。这套模型是为了修一个真实缺陷而引入的（见 10.1 缺陷 D-02）。

6.5 结论生成与横向能力

引擎除了逐条状态，还负责生成三类「人话结论」，全部由确定性规则拼装，模板化输出：

产物	生成方式	示例
汇总结论 headline	三档：硬性全 pass → 大概率可以申；存在 fail → 暂不符合；存在 unknown → 还不能下结论	「还不能下结论：有 13 项条件需要你补充信息或人工确认」
材料清单	按条件命中情况筛选 `materials[]`（只列申请人当前路径需要交的）	「其他方面表现非常突出的详细证明材料（须经学校审核盖章确认）」
接下来做什么	待补充条件 → 「确认 XXX」；时间节点 → 「留意 XXX（截止日期）」	「确认『二年级及以上年级本专科学生』：你的情况里缺少这一项」

横向能力则把同一套引擎复用到更大范围：**政策雷达**（26 项画像 → 规则表映射 → 逐份判定 → 四层分层）、**年度申报日历**（60 个节点 → 解析月份/责任方 → 月度聚合）、**政策检索**（模糊匹配名称/发布单位 + 原文命中）。 三者都不调用模型，全部是确定性逻辑，因此可以像普通函数一样被测试覆盖（测试文件见 8.1）。

07 产品页面与视觉体系

> 这个项目要回答的问题是「我能不能申、还差什么、什么时候申」，所以界面结构不是按技术模块切，而是按**用户的一次完整决策过程**切： 先横向摸底，再纵向核对，最后落到材料与时间。

7.1 信息架构：8 个路由

路由	页面	代码行	核心职责
`/`	首页	197	一句话说清产品、4 个数据统计、六步链路、三个入口（开始匹配 / 政策雷达 / 申报日历）
`/policies`	政策库	344	32 份政策卡片，14 个分类筛选 + 模糊搜索（含原文命中），每页 8 条分页
`/policies/:id`	政策详情	479	条件树、材料、时间节点、原文分段、同类推荐，一页读完不做判定
`/match/:id`	判定页	345	动态表单 + 逐条判定 + 结论卡 + 材料清单 + 时间节点 + 原文高亮
`/radar`	政策雷达	583	26 项共性画像 → 32 份政策四层分层（初筛适合 / 差几项 / 信息不够 / 不符）
`/calendar`	申报日历	438	60 个时间节点按月聚合，标责任方，区分「个人要办」与「学校流程」
`/login`	登录注册	156	单栏表单，演示账号一键填入
`/account`	个人中心	489	判定记录、填报进度、画像概览、本地数据清除（需登录）

表 7-1 8 个路由。前 6 个无需登录即可完整使用——这是刻意设计：**把注册门槛放在用户已经获得价值之后**。

7.2 三条主线的页面配合

横向：先看我适合什么
首页 → 政策雷达。用户只填 26 项共性信息，一次拿到 32 份政策的粗分类， 再点进「去核对材料」进入具体政策。解决「不知道从哪份看起」。

纵向：我到底够不够格
政策库 → 判定页。表单字段由政策动态生成，只问这份政策真正用得到的；提交后左侧表单/右侧结论双栏对照，每条结论可回链原文段落。

时间：什么时候动手
申报日历。把 32 份政策里的 60 个节点摊进一年 12 个月， 标出责任方与「往年一次性节点」，避免用户把学校流程当成自己要办的事。

三个入口共享同一份政策数据与同一套判定引擎，但页面职责严格分开：**雷达不做精细判定，判定页不做时间规划，日历不做资格判断。** 这样每个页面都只有一个主角，不会互相稀释。

7.3 组件设计：三个复用组件

组件	行数	设计要点
`ProfileForm.vue`	189	按 `fields[].group` 分组渲染 text/number/boolean/select/multiselect 五种控件； number 带单位后缀（年、%、人），select 支持「请选择」空态；受控组件 + 统一的 `v-model:profile`
`VerdictReport.vue`	223	结论卡（三档配色）+ 四状态计数 + 通道卡片 + 逐条判定（默认展开不通过的）+ 材料清单 + 时间节点 + 接下来做什么
`PolicyText.vue`	126	渲染 `originalText` 分段，把判定命中的 `quote` 用 `mark` 高亮； 段落按 `sections[]` 折叠，点击「在原文中查看」跳转到对应段落

表 7-2 三个组件被 4 个页面复用。表单、结论、原文三块逻辑各自收敛在一处。

7.4 双主题：一套语义 token，两个色板

早期版本把颜色写死在组件里，改深色主题时要逐个页面改 `bg-white`/`text-slate-900`，漏改就会出现「深色页面里一块白卡」。 后来统一改成**只用语义类名**：组件写 `bg-surface`/`text-ink`/`border-line`/`text-on-brand`， 颜色值集中在 CSS 变量里，切换主题只换变量。

语义 token	青绿浅色（默认）	午夜 AI 深色
`--brand`	#0f6f5c（墨绿，政务可信感）	#38bdf8（天蓝，科技感）
`--accent`	#c2410c（朱红，用于「注意事项」）	#818cf8（靛紫，用于强调）
`--bg`	#f7f8f3（米白）	#080c16（深空）

语义 token	青绿浅色（默认）	午夜 AI 深色
`--surface`	#ffffff（卡片）	#0f1522（卡片）
`--ink`	#14201c（正文）	#e6edf7（正文）
`--line`	#dfe5da（描边）	#1f2b3d（描边）
状态色	pass 绿 / fail 红 / unknown 琥珀 / warn 橙	同语义，降低饱和度并提高亮度以适配深底

表 7-3 两套色板共享同一套语义 token。主题偏好存在 `localStorage`，首屏由 `theme.ts`（52 行）在挂载前同步 `<html class="dark">`，避免闪烁。

> **设计取舍** **为什么保留两套主题，而不是只做一套**
>
> 深色不是「好看」，而是这个产品的场景需要：这类用户常常在晚上、在手机上翻政策。深色主题同时是一次**工程约束的检验**——只有当所有颜色都收敛到语义 token 时，才可能一键切换且不漏底色。事实上第一次做深色主题时，登录页的 tab 高亮用的还是裸颜色， 切到深色后立刻露白，这个缺陷记录在 10.2。

7.5 响应式、可达性与打印

- **布局**：判定页在 ≥1024px 为「表单 380px + 结论自适应」双栏，<1024px 折叠为单栏并允许在「表单/结论」间切换，避免手机上表单过长。
- **表格与卡片**：政策卡片用 `grid` 自适应列数（1/2/3），徽章行允许换行——窄屏下曾出现徽章被挤断行，已改为 `flex-wrap` 修正。
- **键盘**：判定页支持 `Ctrl + Enter` 直接判定；表单控件均为原生 `input/select`，默认支持 Tab 与屏幕阅读器。
- **空态与提示**：任何空列表（无判定记录、无筛选结果、月份无节点）都有明确文案与下一步按钮，不留空白页。
- **打印**：结论区使用 `@media print` 去掉导航与按钮，A4 单栏输出，方便学生打印带去资助中心。

08 质量保障与工程实践

QUALITY & ENGINEERING

> 这个项目最怕的不是页面难看，而是**结论错了但看起来很对**。所以质量保障的重心不在 UI 测试，而在两个地方： 判定引擎的单元测试，以及「数据与原文是否对得上」的契约测试。

8.1 测试分层：9 个文件 / 79 个用例

测试用 Node 原生 test runner（`tsx --test`）运行，不引入 Jest/Vitest，启动快、无额外配置。9 个文件的分布刻意按「一个失败模式一个文件」组织：

测试文件	用例	它守的是什么
`matcher.test.ts`	12	10 种操作符语义、空值 = unknown、建议项不满足只 warning、模板比较、归一化比较
`channel.test.ts`	6	并列通道：任一通道满足即算通过；通道外条件仍需全部满足；全 unknown 时的兜底
`policies.test.ts`	8	数据契约：32 份政策结构合法、条件引用存在、字段/材料/节点引用不断链、通道引用的组真实存在
`detail.test.ts`	8	政策详情聚合：条件树、原文分段、段落偏移、引文在原文中可定位
`radar.test.ts`	8	26 项画像与规则表映射：四层分类、缺信息归入「信息不够」、布尔与 select 语义
`calendar.test.ts`	8	日期解析（「每年 10 月 31 日前」「6 月底」）、月份归属、责任方判定、过期节点不进日历
`search.test.ts`	9	模糊匹配（名称/发文单位）、原文命中、大小写与空格、空查询
`auth.test.ts`	12	注册校验、scrypt 加盐、令牌签发与过期、无效令牌、重复注册
`store.test.ts`	8	用户数据读写：判定记录上限 20 条、进度按政策覆盖、并发写入不丢数据

表 8-1 9 个测试文件共 79 个用例。断言以「行为」为主，例如「留空的硬性条件返回 unknown 而不是 fail」。

运行方式与耗时：`pnpm --filter server test`，全部 79 个用例在 1 秒内跑完，因此可以在每次提交前无负担执行。

8.2 契约测试：把「AI 可能错」变成「结构必须对」

`policies.test.ts` 是整套测试里最特别的一个：它不测函数，只测**数据本身**。断言项包括：

结构与引用完整性

- 每份政策 9 个必填字段齐全，`level`/`category` 在枚举内
- 每条条件的 `docRef` 必须能在 `sections[]` 中找到
- 公式里的字段路径必须在 `fields[]` 中声明
- `channels[].groupIds` 引用的条件组必须真实存在
- 材料、时间节点、条件三者之间不允许出现悬空引用

原文可回溯

- 每条条件的 `quote` 必须是 `originalText` 的**连续子串**（逐字，非模糊匹配）
- 段落偏移 `start/end` 必须与原文切片一致
- 清洗后的原文不得为空，且长度在合理区间
- 每份政策必须有 `review.humanReviewed = true`

第二条是关键：**一旦模型「改了原话」或人工复核手抖改了引文，测试立刻失败**。这把「AI 不编造」从提示词层面的期望，变成了构建时必须通过的硬校验。

8.3 构建与发布流水线

#	步骤	命令 / 产物
1	类型检查	`pnpm --filter server typecheck`、`pnpm --filter web typecheck`（均需 0 错误）
2	单元与契约测试	`pnpm --filter server test` → 79 passed
3	导出静态数据	`pnpm --filter server exec tsx scripts/export-static-data.mjs` → `web/public/static-data/*.json`
4	前端构建	`pnpm --filter web run build:pages`（`base=/shuanghuiwangzhongwang/`，`VITE_STATIC_DATA=true`）
5	发布到 gh-pages	`node tools/deploy-pages.mjs`：用 GitHub Git Data API 建 tree → commit → 快进 gh-pages 分支

#	步骤	命令 / 产物
6	同步 main	`node tools/git-sync.mjs`：同上机制推 main，含网络抖动与 422 瞬态重试

表 8-2 一次完整发布的 6 个步骤。任一步失败即中止，不会发布半成品。

两个 `tools/*.mjs` 的存在有特殊背景：本机 `git push` 与 `git fetch` 走网络经常被中断（详见第九章 9.6）， 最终改用 GitHub Git Data API 直接创建 commit 与 tree，再由本地精确重建 commit 对象对齐 SHA。这是被工程约束逼出来的方案，但也带来一个额外好处：**发布脚本可以精确控制「哪些文件进 gh-pages」**，不需要维护额外分支的工作区。

8.4 验收方式：每个里程碑都在浏览器里点过

本项目没有引入 E2E 测试框架（成本与收益不匹配：页面逻辑大多由纯函数驱动，E2E 覆盖不住判定正确性）， 但每个功能里程碑都执行同一套人工验收清单：

- **走通主链路**：从首页 → 选政策 → 填表 → 判定 → 看原文 → 回列表，全程无报错、无空白。
- **边界场景**：全空表单提交、只填一项、全部填「否」、数值填 0 与极大值、中文与符号混输。
- **静态版与在线版分别验**：静态模式 `VITE_STATIC_DATA=true` 走浏览器内判定；在线模式走 `/api/policies/:id/evaluate`。两者结论必须一致。
- **部署后回访**：gh-pages 发布后验证深链（直接打开 `/match/national-scholarship`）与资源 200。

8.5 版本管理与协作约定

仓库根目录的 `AGENTS.md` 写死了三条铁律，整个开发过程按它执行：

① 重大更改必提交
新功能跑通、依赖/架构/数据模型变更、主流程 bug 修复、视觉或部署大改 → 必须 commit 并同步 GitHub。 文案与样式微调不算重大，可合并提交。

② main 永远可运行
任何提交后 main 都必须能起服务、能通过测试；大改走 `feat/xxx` 分支，验证通过再合并。

③ 细粒度历史
一次提交只做一件事，提交信息用 `feat/fix/docs/style/refactor/test/chore` 前缀 + 一句中文说明。 比赛的评审会看提交历史，一次性提交全部功能会被判为开发过程不可信。

实际执行结果：一次提交只做一件事，最大单次改动是「政策库扩充到 31 份」（新增 19 份政策数据），其余每次改动都能对应回一个具体问题或一个页面。 每个里程碑打一个 tag，回档时 `git reset --hard <tag>` 即可恢复可用状态——这也是用户明确要求的能力：「不要让网页因为乱改崩溃还没有办法回档」。

09 重大版本更新

MILESTONES

项目从空仓库走到可演示成品，经历五个阶段：骨架与引擎、政策库扩充、账号与静态化、横向能力、打磨与纠偏。 这一章按里程碑回放「每次重大更新解决了什么问题、产生了什么」，而不是流水账式列出所有提交。

9.1 演进总览

阶段	主题	政策	页面	关键产出
M0	仓库与协作约定	0	0	AGENTS.md 铁律、回档 SOP、GitHub API 同步工具
M1	骨架 · 引擎 · 首批政策	5	3	条件树模型、判定引擎、抽取脚本、数据契约测试
M2	账号 · 扩容到 32 · 静态化	32	5	账号体系、政策库筛选/搜索、纯静态部署能力
M3	gh-pages 在线演示	32	5	可公开访问的演示地址、深链回退
M4	雷达 · 日历 · 详情页 · 双主题	32	8	横向初筛、时间规划、单政策全景页、深色主题
M5	体验重构与搜索增强	32	8	模糊搜索、原文命中、首页主入口重构
M6	一次被完整回滚的尝试	32	8	口语化改造 → revert，验证回档能力

表 9-1 六个里程碑。政策数从 5 到 32，页面数从 3 到 8，测试从 8 到 79 个用例。

9.2 M1：把「解释政策」变成「编译政策」

这是整个项目最关键的一段，确立了后面所有工作的地基。按提交顺序有四步：

① 领域模型先于界面

`93c54de` 先写 `types.ts`（条件树 schema）与 `matcher.ts`（纯函数引擎），配 8 个单测覆盖缺失字段、any 组、边界值。 此时还没有任何页面——先用测试把「判定语义」钉死，界面才不会被数据牵着走。

② 首批 5 份政策踩了一个坑

`d5bf8fa` 抓的第一批里有《南京租房补贴》——地方性政策，与「全国普适」定位冲突。 `a3b4a6d` 立刻把原文统一替换为 5 份全国性政策，并留下 `_sources.json` 记录来源与抓取时间。

选题定位在这一步被纠正。

③ 抽取链路与「证据链」同时成型

`9fa9fd1` 完成 5 份政策的 AI 抽取 + 人工复核，同时引入 `review` 字段与 `apply-review.mjs`； 第一次处理过时条款（助学贷款 6000 元额度、5 个试点省份）。 `6f0267d` 紧跟一步写出数据契约测试，把「quote 必须可回溯」变成构建门槛。

④ 引擎与前端第一次对接就暴露结构差异

`abb040f` 出 API，`a12520e` 出页面，随即 `ff68877` 修扁平键画像兼容（详见 D-01）。

「设计先行、暴露即修」的节奏从这里开始固定下来。

9.3 M2：从「能判一份」到「能当产品用」

这个阶段提交最密集（约 28 个），有三条主线并行推进：

主线	做了什么	为什么是这一步
账号体系	`70227b5` 注册/登录/令牌鉴权/个人数据；`f2baf3e` 登录页与个人中心；`e938ee2` 草稿与判定记录按账号同步	用户换设备（机房→宿舍）时不想从零填起；但填写内容本身不保存，只存进度与记录
政策扩容	8 份（`3fc9b1b`）→ 12 份（`ce81cb2`）→ 31 份（`f2a01bf`，一次 +19）→ 32 份（`0c04264`）	32 份覆盖从学前到研究生、从奖助到社会救助的全学段；一次扩 19 份是效率最高的一步
静态化	`af10133` 判定引擎可在浏览器本地跑 → `ff35f16` 发布脚本 → `717c3af` 在线演示	在线演示是比赛加分项，且能让评审不装环境就看到成品

同时做了两个**产品口径上的重要决定**，它们不是功能，但影响全部页面：

取消「需人工确认」状态（`e61820f`）

填写内容不再保存（`11f3a55`）

早期对「家庭经济困难」这类主观条件统一标为「需人工确认」，结果所有政策都被这条占据， 用户看不出自己到底还差什么。改为**把主观条件也做成可填字段，由用户自填是/否，引擎照常判定**——把「无法判定」变成「请回答」。

原先判定页会把表单存进服务端，后来判断：这类数据敏感、且用户不需要「下次接着改」， 于是改为**只存在页面内存，离开即清空**，服务端只记「判定过哪份政策、填了几项」。这是隐私与体验的双赢。

9.4 M3：把成品搬到公网

静态部署不是「打个包上传」，而是三个连锁问题：

- **判定要在浏览器里跑**：服务端 `/api/policies/:id/evaluate` 在静态站不存在，于是前端加 `isStaticMode` 分支， 静态模式下直接调用同一个 `matcher.ts`（通过 Vite 别名 `@domain` 复用 server 的领域代码，**不是复制一份**）。
- **数据要变成静态 JSON**：`export-static-data.mjs`（94 行）把政策库、雷达、日历全部导出到 `web/public/static-data/`。
- **项目页有子路径**：站点部署在 `/shuanghuiwangzhongwang/`，路由 base 与资源路径都要带前缀， 否则一刷新就 404（详见 D-13）。

9.5 M4：三个新页面 + 一次系统性纠偏

这一阶段的产出密度最高，新增三个页面：

新增	解决的问题	实现要点
政策雷达 `/radar`	32 份政策逐份填不现实，用户不知道该看哪份	26 项共性画像 + 规则表映射 285 种字段写法，四层分层；缺失信息只标「信息不够」
申报日历 `/calendar`	知道够不够格之后，不知道什么时候动手	纯规则解析 60 个节点（不调用模型），区分固定/非固定节点，标责任方
政策详情 `/policies/:id`	政策库只有卡片，原文与条件树 90% 没展示	条件树格式化（算子转中文）、五段正文、同类政策推荐打分

同一阶段还完成了**双主题改造**（`03e5bb8`）：把全站写死的 `slate/white` 类名迁移到语义 token，新增 `theme.ts` 管理切换与记忆。 以及**个人中心**（`195fa7d`）：汇总判定记录、填报进度、画像概览。

> 关键纠偏 **提交 `b0caaca` ：一次修复四类「看起来对但错」的问题**
>
> 本阶段末尾的 `b0caaca` 是整个项目最重要的一个修复提交，一次处理了四件事：**多通道判定被错误叠加成 AND**（D-05）、 **入学年份与年限被直接比较**（D-04）、**雷达四层命名误导**（D-10）、**日历缺责任方导致用户误把学校流程当自己的待办**（D-11）。 测试从 54 个扩到 70 个。这些问题的共同点是：**功能都「能用」，但结论会骗人。**

9.6 M5：体验重构与搜索增强

- **入口重构**：首页主按钮从「开始匹配」改为「政策雷达·横评 32 份」，标题改为「填一次自己的情况，30 秒知道哪些钱能申」。 理由是真实使用路径：**先用雷达定位，再进单份政策**，而不是让用户先猜一份政策。
- **搜索增强**（`90ca2b2`）：原先只做严格子串匹配，「困难认定」搜不到「家庭经济困难学生认定」； 改为分词式模糊匹配 + 原文命中，并按「原文命中」标注结果来源。
- **字段细节**：残疾等级补「没有残疾」选项与「选填」标记——此前用户没有残疾证时被迫留空，引擎判为「待补充」，体验很别扭。

9.7 M6：一次被完整回滚的尝试

回滚这一次的提交是一次**从上线到回滚、12 分钟内完成的完整回环**，值得单独记录：

12:00 前后 • 改造

`8166d8d`：把政策表单的所有选项改成口语化表述（例如「本科或专科（含第二学士学位、预科生）」改为更口语的说法）， 涉及 30 个文件、234 处改动，原文与判定逻辑不变。

上线验收

测试 79 个用例仍全过、类型检查通过、静态部署成功。技术上这次改动是「正确」的。

00:15 • 回滚

用户反馈「还是改回原来的吧」。执行 `git revert --no-edit 8166d8d`（`6b13d6c`），`git diff --stat 90ca2b2 HEAD` 输出为空——

文件树与改造前逐字节一致

。随后重新测试、重新部署、验证线上恢复原选项。

这次回滚验证了三件事：① `git revert` 比手改回去可靠，因为它保留了「为什么改回去」的历史；② 回档后的状态可以用 diff 客观验证（空 diff），而不是靠感觉；③ 比赛要求的「完整开发过程」不排斥回滚——**回滚本身也是一次真实的工程决策，值得留在历史里。**

10 缺陷修复记录

BUG FIXES

9 个 `fix` 提交 + 1 次 revert 看起来不多，但**被修复的问题里有 5 个会让用户拿到错误结论。** 这一章按「严重程度」而不是「修复时间」组织，重点讲清每个缺陷的根因与防回归手段。

10.1 A 类：会导致错误结论的判定缺陷

这类缺陷的共同特征是：**页面正常渲染、没有报错、测试也全绿，但结论是错的。**它们只能靠「读原文 + 人工核对判定结果」发现， 因此每一个修复都补了对应的单测。

编号	现象	根因	修复	防回归
D-01	服务端判定正常，从页面提交却整份政策「全部待补充」	前端表单产出扁平键（`identity.nationality` 作为字面 key）， 引擎按点路径下钻嵌套对象，`getField` 取不到值	`getField` 先查扁平键、再按点路径下钻，两种画像结构都能判定	新增「扁平键画像驱动真实政策判定」用例 + 两种结构一致性用例
D-02	同时显示「0 项不符合」却给出「暂不符合」结论	条件组 `logic=any` 时按「硬性条件集合」判断是否通过； 若组内条件全为 `required=false`，硬性集合为空 → 整组判 fail	先剔除建议项，规则改为「任一条 pass 即 pass；否则有待补充即 unknown；只有明确不满足才 fail」	`channel.test.ts` 覆盖「全建议项 any 组」
D-03	符合条件的学生被判「暂不符合」	入学门槛按 **字段值直接比较**：把「入学年份 2024」与「年限 5」相比	改为按**年级**判断（2 ≤ 年级 ≤ 5），入学年份只用于推算年级	新增年级语义用例；判定页增加「年级」为显式字段
D-04	国家奖学金对所有人判不符合	「常规成绩通道」与「突出表现通道」是**或**关系，抽出后被写进同一组用 all 合并 → 要求同时满足	引入 `channels` 并列通道模型：通道内仍需满足，通道之间取任一，通道外条件组照常全部满足	`channel.test.ts` 6 个用例 + 3 份政策的数据契约断言
D-05	少数民族骨干计划对任何人都不符合	「报考硕士（年龄 ≤40）」与「报考博士（年龄 ≤45）」同处一个 all 组——不可能同时是硕士和博士	拆成两条并列通道（`g3a` / `g3b`）	同上；并在复核清单中记录该次修正
D-06	布尔条件理由显示「你的情况是 true」	引擎直接输出原始布尔值，未转成用户语言	理由渲染统一为「是 / 否 / 未填写」	判断结果文案由 `verdictOf()` 统一生成，页面不再自行格式化

表 10-1 A 类缺陷 6 个。修复全部落在领域层，页面无需改动即可受益。

深度剖析 **为什么「逻辑组合」是这类系统最大的风险**

D-04 与 D-05 是同一类错误的两个变体：**单条条件都忠实于原文，组合方式却错了。**模型被要求「逐条抽条件」， 它确实做到了；但「这些条件之间是且还是或」需要理解整体语义，而这恰恰是提示词最难约束的部分。

更麻烦的是，**这类错误在数据层面完全合法**：JSON 结构正确、docRef 有效、quote 可回溯、测试全绿。它只会在「拿一个真实的人去跑一遍」时暴露。 这就是为什么项目必须做两件看起来「笨」的事：**每份政策抽完人读一遍原文**；**每个里程碑用真实场景点一遍全流程。** 目前 3 份政策、6 条通道的并列结构，全部来自这种人工核对。

10.2 B 类：数据与证据链缺陷

编号	现象	根因	修复
D-07	AI 返回的段落文本与原文对不上，`indexOf` 定位失败，原文高亮失效	原文含换行与全角空格，模型输出的段落常把空白规范化，直接按字符匹配必然失败	改为「去空白后匹配 + 索引映射回原文」，并配套 `repair-sections` 按原文补齐被截断的段落
D-08	部分条件在原文中搜不到（引文跨行被切断）	人工复核时手工复制引文，跨行处少复制了半句	修复基层就业代偿两处跨行引文；数据契约测试断言「quote 必须是 originalText 的连续子串」，这类错误从此过不了构建
D-09	三处段落文本无法回溯	抓取时截断了段落尾部，条件引用了被截断的内容	重新抓取全文并重算段落偏移（`f2a01bf` 同期修正）

编号	现象	根因	修复
D-10	`data/policies/raw/` 被 gitignore 排除，仓库里没有原文	`.gitignore` 的 `data/` 规则过宽，连政策库一起忽略	收窄为只忽略运行期数据（SQLite、上传缓存），原文与来源清单入库
D-11	「需人工确认」占据大量条件，用户看不出真正差什么	把主观条件当成「引擎无法判定」处理	取消该状态，主观条件改为可填字段由用户自答；状态文案统一为「待补充」（`e61820f`、`a153c58`）
D-12	残疾等级没有「没有残疾」选项	表单选项直接取自模型抽取的枚举值，缺「无」这一档	补「没有残疾」选项并标记为选填（`90ca2b2`）

表 10-2 B 类缺陷 6 个。D-08 之后，同类问题从「靠人复查」升级为「靠测试拦截」。

10.3 C 类：工程与部署缺陷

这类缺陷不影响结论正确性，但会让「演示当场翻车」，因此修复优先级同样很高。

编号	现象	根因	修复
D-13	gh-pages 上直接打开 `/match/xxx` 刷新落到 GitHub 404 页；或刷新后被重定向回首页	① 静态站没有服务端路由回退；② 项目页部署在子路径，路由 base 未带前缀	发布脚本额外产出一份内容同 `index.html` 的 `404.html`；`createRouter` 传入 `import.meta.env.BASE_URL`
D-14	`git-sync` 偶发失败：GitHub 建 tree 后提交返回 `422`	GitHub 侧对刚创建的 tree 存在可见性延迟，紧接着的 commit 请求会拿到 422	对 422 也纳入瞬态重试（`8e2bab1`），并覆盖「建树后提交」这一阶段（`ba53044`）
D-15	`git-sync` 因网络抖动直接终止	把 `fetch failed`（连接重置 / DNS 波动）当永久错误	网络异常按瞬态错误重试 4 次（`96c88ef`）
D-16	编辑政策 JSON 时 dev server 崩溃，报 `EBUSY`	Vite 文件监听与 Windows 文件锁冲突	加固 `server.watch` 配置（`3077327`），忽略 `data/*.json` 的热更新触发
D-17	本地重建的 commit 与 GitHub 返回的 SHA 不一致，分支无法对齐	commit 对象文本的差异（message 尾换行、时区写法 `+0800` vs `+0000`）会导致不同 SHA	`git-sync` 遇 SHA 不一致时，按远端元数据**穷举变体重建本地对象**，命中后再对齐 `refs/heads/main`（`aabbe50`）
D-18	搜索「困难认定」搜不到「家庭经济困难学生认定」	只做严格子串匹配	改为分词式模糊匹配 + 原文命中，并标注命中来源（`90ca2b2`）

表 10-3 C 类缺陷 6 个。D-14 与 D-15 让同步脚本从「能跑通」变成「能连续稳定运行」。

10.4 D 类：界面与体验缺陷

编号	现象	修复
D-19	深色主题下登录页 tab 选中态露白、部分按钮仍是白底黑字	全站颜色收敛为语义 token，选中态加深青底、按钮改深板浅字（`03e5bb8`）
D-20	窄屏下顶栏按钮挤成两行、后台连接徽章占位	窄屏隐藏连接徽章，按钮组允许换行并收紧间距（`03e5bb8`）
D-21	雷达四层命名「能申 / 差一点 / 还缺信息 / 不符合」被理解成审核结论	改称「初筛适合 / 差几项 / 信息不够 / 不符」，并写明「还有几项要进政策页核对」（`b0caaca`）
D-22	日历把学校/主管部门的流程节点与个人待办混在一起，用户以为都要自己办	每个节点标责任方（个人要办 / 学校院系 / 主管部门），新增「只看个人要办」筛选（`b0caaca`）

表 10-4 D 类缺陷 4 个。D-21 表面是文案问题，实质是**降低用户对结论的期待值**，属于必要的诚实。

10.5 修复方法论：四步闭环

22 个缺陷（D-01 ~ D-22）不是随机修的，整个过程收敛成一套固定动作：

① 用真实场景复现
每条缺陷都对应一个具体的「用户路径 + 输入 + 错误输出」，例如「本科二年级、排名 5%，国奖被判定不符合」。复现不出来的一律不当作缺陷处理。

② 定位到领域层还是展示层：先确认是引擎算错，还是页面显示错。前者必须补单测，后者必须收敛到统一的格式化函数。

③ 修复 + 补测试 + 数据检查
代码修完后，**回头检查已有 32 份政策是否都受同一问题影响**——D-04 修复时就顺带发现了中职国奖与少数民族骨干计划两处同类问题。

④ 记录成文档：每个缺陷的根因写进提交正文与本文档，避免「同样的错换个地方再犯」。回滚也走同样的流程（`revert` + diff 验证 + 重新部署）。

11 已知边界与后续规划

LIMITS & ROADMAP

> 一个诚实的工程文档必须写清「什么没做、什么做不到、什么做了但有局限」。这一章把边界摊开， 既是为了避免用户误用，也是为了给后续迭代留下明确的起点。

11.1 已知边界

边界	具体表现	为什么现在这样
只覆盖国家级政策	32 份均为全国性文件；省市级实施细则、校级评审办法不在库内	地方文件数量级大、时效差异大、格式更不规范；先把「全国通用骨架」跑通再谈扩展
政策会过期	库内文件年份跨 2005–2025，部分额度与范围已被新文件替代	已在 notes 标注【现行标准】并在判定页展示；但无法保证实时同步官方更新
静态版无账号	gh-pages 上不能注册登录，判定记录与进度不跨设备	静态站没有服务端；演示与真机使用走同一套判定引擎，仅账号能力缺失
不做图片识别	不能拍照识别证书、成绩单	所用模型（DeepSeek）当前不支持图像输入；强行做会引入不可控的识别误差
主观条件靠自述	「家庭经济困难」「品行端正」由用户自己填是/否	这类判断本就只能由学校评审环节认定，产品只能如实呈现，不能替用户下结论
数据存储为单机 JSON	账号与用户数据存在 `data/*.json`，无数据库、无并发控制	面向演示与本机使用，量级在百人内足够；换成 SQLite/Postgres 是明确的后续项
无端到端自动化测试	页面交互依赖人工验收清单	判定正确性由领域层单测覆盖，E2E 收益低；若后续页面复杂化再引入 Playwright

表 11-1 七条已知边界。每一条都在页面上有对应的用户提示，不靠文档单方面声明。

11.2 已知技术债

- `radar.ts` 顶部注释仍写「21 个抽取槽位」，实际已扩到 **26 项**（`RADAR_SLOTS` 与界面均为 26）——注释未同步。
- README 中「70 个单元测试」的表述已过时，当前为 **79** 个用例。
- 提交历史里有 2 条通过 GitHub 网页直接修改 README 产生的提交（`1a6ac60`、`aeef746`），未走 `type:` 前缀规范。
- 时间解析（`parseDeadlineText`）基于正则规则，能覆盖「每年 X 月 X 日前」「X 月底」「X-X 月」「每学年」等常见表述， 但遇到「以当年通知为准」「9—10 月」这类模糊表述会归入「其他流程节点」而非硬解析——这是有意保守，不是遗漏。
- 界面文案中仍有少量「大概率可以申」等口语化措辞，属于产品语气选择，但需在文档中明确其**非承诺**属性。

11.3 后续规划

优先级	方向	具体内容
P0	数据时效保障	为每份政策记录「官方上次核对日期 + 下一次复核期限」，超期政策在列表与详情页显式降权提示
P0	地方/校级扩展	按「省级通用条件 + 校级补充条件」两层建模，让用户填一次画像即可叠加本地细则
P1	账号与存储升级	JSON 文件 → SQLite，补并发写与迁移脚本；静态版保留本地存储降级方案
P1	材料清单可勾选与导出	材料清单加勾选进度与 A4 打印/PDF 导出，形成「去办事前的一张纸」
P1	政策对比	2–3 份政策并排比较条件差异，回答「国奖 vs 励志，我更适合哪个」
P2	抽取流水线工程化	把「抓取 → 清洗 → 抽取 → 校验 → 人审」做成一条可重跑的命令，支持增量新增政策
P2	端到端测试	引入 Playwright 覆盖 5 条主链路，把人工验收清单自动化

表 11-2 后续规划按「对结论可信度的影响」排序，而不是按实现难度排序。

11.4 结语：这个项目到底做了什么

如果只用一句话概括：**它把「读政策」这件需要经验和耐心的事，拆成了「AI 编译 + 规则判定 + 原文回链」三段，并让每一段都可检查。**

对用户，价值是具体的：填一次表，看到自己与目标政策的差距在哪、还缺什么材料、下个月该做什么；每一条结论都能点开原文自己核对。

对比赛评审，这个项目刻意展示的是**工程判断力**而非功能数量：为什么把大模型限制在建库阶段、为什么愿意花 57 条记录做人工复核、 为什么一次修复要回头检查 32 份政策、为什么功能改坏了要选择回滚而不是硬着头皮改。这些选择都写在提交历史与本文档里。

一句话总结 **让 AI 只在最需要它的地方出现一次，然后让确定性的部分足够可靠。**

32 份政策、339 条条件、79 个测试、0 次运行期模型调用——这组数字构成了这个项目的骨架。

附 附录

APPENDIX

A 接口清单（20 个端点）

后端为 Express，全部挂在 `/api` 前缀下。**带「鉴权」标记的接口需要 `Authorization: Bearer <token>`。**

方法	路径	鉴权	说明
GET	`/health`	—	健康检查，返回 `{ok:true}`
POST	`/auth/register`	—	注册：账号 3–20 位字母数字下划线，密码 ≥6 位
POST	`/auth/login`	—	登录，返回无状态令牌
GET	`/auth/me`	✓	当前登录用户信息
POST	`/auth/logout`	—	前端清除令牌（服务端无会话）
GET	`/policies`	—	政策列表（只返回列表所需字段）
GET	`/policies/:id`	—	政策详情：条件树、原文、材料、节点、复核记录
POST	`/policies/:id/evaluate`	—	**核心判定接口**：body `{profile}` → 逐条状态 + 结论 + 建议
GET	`/calendar`	—	申报日历：60 个节点按月份聚合
GET	`/radar`	—	雷达元数据：26 项画像槽位 + 映射规则
POST	`/radar/scan`	—	雷达扫描：body `{profile}` → 32 份政策四层分层
GET	`/me/drafts/:policyId`	✓	读取某份政策的填报进度（只存已填项数）
PUT	`/me/drafts/:policyId`	✓	写入/更新填报进度
DELETE	`/me/drafts/:policyId`	✓	删除单条进度
GET	`/me/records`	✓	判定记录列表（最多 20 条，时间倒序）
POST	`/me/records`	✓	追加一条判定记录
DELETE	`/me/records`	✓	清空判定记录
GET	`/me/progress`	✓	全部填报进度汇总
PUT	`/me/progress/:policyId`	✓	更新进度
DELETE	`/me/data`	✓	**一键清空**当前账号全部数据（判定记录 + 进度）

表 A-1 接口清单。`/policies`、`/calendar`、`/radar` 三组在静态模式下由预导出 JSON 替代。

B 政策数据字典

字段	类型	说明
`id`	string	政策唯一标识（英文短横线命名），同时作为静态 JSON 文件名
`title` / `issuer`	string	政策全称 / 发布单位
`level` / `category`	string	层级（国家级）/ 类别（奖学金、助学金…共 14 类）
`year` / `sourceUrl`	number / string	发布年份 / 官方原文链接
`sections[]`	array	原文分段：`{id, heading, text, start, end}`，偏移用于原文定位
`fields[]`	array	需填写字段：`{key, label, type, group, options?, unit?, required?}`
`conditions[]`	array	条件组：`{id, name, logic:'all'|'any', items[]}`
`conditions[].items[]`	array	单条条件：`{id, field, label, operator, value, required, docRef, quote, hint}`
`channels[]`	array?	并列通道：`{id, name, description, groupIds[]}`，通道之间满足任一即可
`materials[]`	array	申请材料：`{name, note, required, conditionId?}`，可按条件筛选
`deadlines[]`	array	时间节点：`{name, date, note, owner, kind}`，`owner` 区分个人/学校/主管部门
`notes[]`	array	提示与【现行标准】说明，判定页折叠展示
`extraction`	object	抽取留痕：`{engine, model, at}`

字段	类型	说明
`review`	object	人工复核：`{humanReviewed, reviewedAt, reviewer, changes[]}`
`cleaning`	object?	清洗记录：`{removedChars, removedFrom[]}`，仅 7 份有
`originalText`	string	清洗后的政策原文（条件 quote 必须能在其中逐字定位）

表 B-1 政策 JSON 顶层结构。完整类型定义见 `server/src/domain/types.ts`（241 行）。

C 提交规范

类型	什么时候用	本项目示例
`feat`	新增一个可用的功能或页面	政策雷达、申报日历、政策详情页、账号体系
`fix`	修正已交付内容中的错误	判定语义、引文偏移、深链回退、同步重试
`docs`	只改说明与文档	README、协作约定、部署说明、演示账号说明
`style`	只改样式与文案，不动逻辑	标题字号、入口按钮、状态文案、首页配色
`test`	只加或改测试	数据契约测试、原文清洗与偏移校验
`refactor`	结构整理，行为不变	登录页单栏化、移除首页常见问题
`chore`	工具、依赖与仓库设置	同步工具、静态导出脚本、初始化仓库

表 C-1 提交类型规范。一次提交只做一件事，提交信息写清「改了什么 + 为什么 + 怎么验证」。

D 复现与验证

```
# 1. 安装依赖（Node.js ≥ 20.19、pnpm ≥ 9）
pnpm install

# 2. 配置环境变量（AI 抽取才需要，浏览与判定不需要 Key）
cp .env.example .env
#   DEEPSEEK_API_KEY=sk-xxxx      # 仅重新抽取政策时使用

# 3. 启动开发环境：前端 5173 / 后端 8787
pnpm dev
curl http://localhost:8787/api/health

# 4. 质量校验（本项目每次提交前执行）
pnpm --filter server typecheck
pnpm --filter web typecheck
pnpm --filter server test        # 79 个用例

# 5. 静态演示版构建与发布
pnpm --filter server exec tsx scripts/export-static-data.mjs
pnpm --filter web run build:pages
node tools/deploy-pages.mjs

# 6. 验证数据契约（不启动服务，只查数据是否正确）
pnpm --filter server exec tsx --test src/domain/policies.test.ts
```

表 D-1 从零复现的完整命令序列。步骤 2 可跳过——不配置 Key 也能完整使用全部判定功能。

本文档由项目仓库 `docs/tech-report/` 下的 HTML 源文件生成，源文件与代码同仓库、同版本， 可与其他提交一起回溯。文档版本 v1.0，对应提交 `6b13d6c`，生成日期 2026-10-27。